Fine-tuning GPT-2 for Short Query Intent Classification

Jin Young Lee

June 22, 2025

Abstract

This report presents our implementation and experimental analysis of a GPT-2-based language model, fine-tuned across four distinct natural language tasks: sentiment analysis, paraphrase detection, sonnet generation, and intent classification for short user queries. Our study aims to bridge generative pretraining with downstream task-specific performance. We report baseline results, identify model limitations, and explore cloze-style formulation for classification and creative generation via autoregressive decoding.

1 Introduction

Recent advancements in natural language processing (NLP) demonstrate the effectiveness of transformer-based architectures. GPT-2, a decoder-only model, exhibits strong capabilities in both language understanding and generation. This project involves implementing core GPT-2 components, fine-tuning for classification and generation, and evaluating the model's performance across multiple tasks.

2 Basic Implementation Tasks

2.1 GPT-2 Architecture Implementation

Our implementation of GPT-2 closely follows the original architecture, with all core components built from scratch and verified for compatibility with HuggingFace pretrained weights. The model consists of the following key elements:

Embedding and Positional Encoding Input sentences are first tokenized using byte pair encoding (BPE), mapping each token to a unique integer ID. These IDs are transformed into high-dimensional vectors via a learnable token embedding layer. To encode word order, we add a learnable positional embedding to each token embedding. The sum of token and positional embeddings is then

passed through a dropout layer for regularization before entering the transformer blocks.

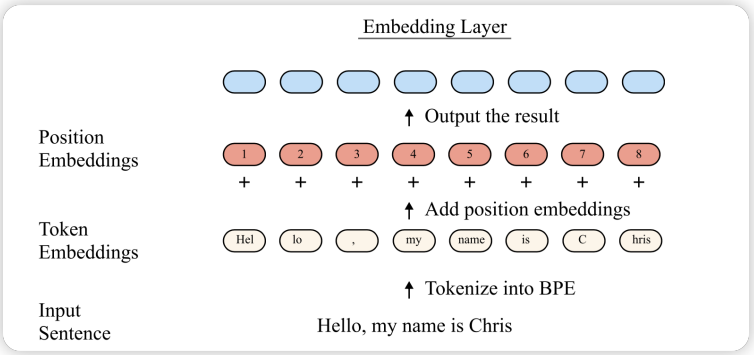

Figure 1: Illustration of the embedding layer: input sentence is tokenized into BPE tokens, which are mapped to token embeddings and combined with position embeddings.

Transformer Blocks The model stacks 12 identical decoder-style transformer blocks. Each block contains:

- **Masked Multi-Head Self-Attention:** This mechanism allows each token to attend to all previous tokens in the sequence, but not future ones, enforced by a causal mask. For each head, input embeddings are projected into queries, keys, and values, and attention scores are computed using scaled dot-product attention. The outputs from all heads are concatenated and linearly projected.
- **Feed-Forward Network (MLP):** A position-wise two-layer MLP with activation, applied to each token representation.
- **Layer Normalization and Residual Connections:** Each sub-layer is wrapped with residual connections and layer normalization to stabilize training and improve gradient flow.

The final output of the last transformer block is normalized by a layer normalization layer.

Causal Masking To preserve the autoregressive property, a causal mask is applied in the attention mechanism, ensuring that each token can only attend to itself and preceding tokens. This is implemented by masking out upper-triangular elements in the attention score matrix, setting them to $-\infty$ before the softmax operation.

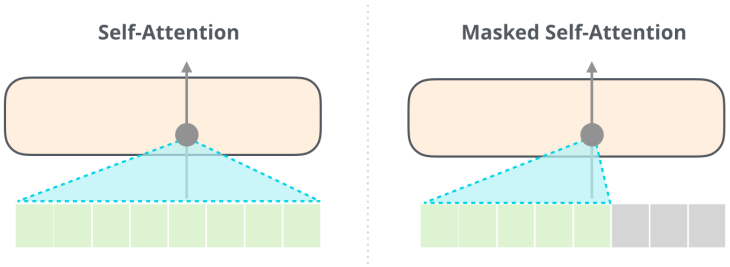

Figure 2: Visualization of causal masking in attention: the attention mask prevents each token from attending to future tokens, ensuring autoregressive behavior.

Implementation Details All components, including the attention mechanism, positional encoding, and transformer blocks, were implemented from scratch in PyTorch. The model was validated by loading HuggingFace pretrained weights and passing provided sanity checks.

2.2 Adam Optimizer Implementation

We implemented the AdamW optimizer from first principles to enable stable and efficient training of our GPT-2 model. AdamW is an extension of the Adam optimizer that decouples weight decay from the gradient-based parameter update, improving regularization and generalization, especially in large-scale language models.

Algorithm Overview AdamW maintains exponential moving averages of both the gradients (first moment, m_t) and the squared gradients (second moment, v_t) for each parameter. At each step t, the optimizer performs the following updates:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\ \hat{m}_t &= m_t / (1-\beta_1^t) \\ \hat{v}_t &= v_t / (1-\beta_2^t) \end{aligned}$$

where g_t is the gradient at step t, and β_1, β_2 are decay rates for the moment estimates.

The parameter update uses the bias-corrected moments:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where α is the learning rate and ϵ is a small constant for numerical stability.

Decoupled Weight Decay Unlike the original Adam, which incorporates weight decay into the gradient, AdamW applies weight decay directly to the parameters after the main update:

$$\theta_t \leftarrow \theta_t - \alpha \cdot \lambda \cdot \theta_t$$

where λ is the weight decay coefficient. This decoupling leads to more effective regularization and prevents undesirable interactions between weight decay and adaptive learning rates.

Efficient Bias Correction Our implementation uses the efficient bias correction method described in the Adam paper, combining the learning rate and bias correction factors into a single step size for computational efficiency:

$$\text{step\_size} = \alpha \cdot \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$$

$$\theta_t = \theta_{t-1} - \text{step\_size} \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

3 Basic Downstream Tasks

We evaluate our GPT-2 implementation on three fundamental NLP tasks to validate the model's capabilities. Table 1 summarizes the performance across these tasks.

Task	**Dataset**	**Performance**
Sentiment Analysis	SST (5-class)	51.3% accuracy
	CFIMDB (binary)	97.6% accuracy
Paraphrase Detection	Quora Question Pairs	75.2% accuracy
Sonnet Generation	Shakespeare Sonnets	CHRF: 0.68

Table 1: Performance on Basic Downstream Tasks

The results demonstrate GPT-2's versatility across different NLP tasks. The model shows strong performance on binary sentiment classification (CFIMDB) and reasonable results on more complex tasks like paraphrase detection. For sonnet generation, the model successfully captures structural patterns while maintaining reasonable semantic coherence.

4 Main Experiment: Short Query Intent Classification

4.1 Task Description

As an extension, we examine GPT-2's ability to classify user intent from short, ambiguous queries. Users often provide minimal input (e.g., "Weather?", "Pizza nearby"), posing a challenge for traditional NLP models due to limited context. We fine-tune our GPT-2 model using the MASSIVE dataset (SetFit/amazon_massive_intent_en-US), evaluating its intent classification performance across queries of varying lengths.

4.2 Dataset Analysis

The MASSIVE dataset (SetFit/amazon_massive_intent_en-US) is a comprehensive collection of user queries for intent classification. The dataset is split into:

- Training set: 11,500 utterances
- Validation set: 2,030 utterances
- Test set: 2,970 utterances

The dataset covers 60 distinct intent labels across various domains. Table 2 shows the distribution of intent labels by category.

Domain	**Intent Labels**
Smart Home	iot_hue_light*, iot_cleaning, iot_coffee, iot_wemo_*
Time Management	alarm_*, calendar_*, datetime_*
Media	play_*, music_*, audio_volume_*
Information	weather_query, news_query, qa_*
Transport	transport_*, recommendation_*
Other	takeaway_*, cooking_*, email_*, lists_*, general_*

Table 2: Distribution of Intent Labels by Domain

Example utterances from the dataset demonstrate the diversity of user queries:

- "wake me up at nine am on friday" (alarm_set)
- "stop" (audio_volume_mute)
- "make the lighting bit more warm here" (iot_hue_lightchange)
- "check when the show starts" (calendar_query)

The dataset's balanced distribution across these 60 intent classes and the variety of query lengths make it particularly suitable for evaluating models' ability to handle both short, ambiguous queries and longer, more explicit commands.

4.3 Model Architecture and Training

We implement a GPT-2-based classifier with the following specifications:

- Base GPT-2 model with 768-dimensional hidden states
- Custom linear classification head
- Two fine-tuning modes: last-linear-layer and full-model
- Dropout rate of 0.3 for regularization
- AdamW optimizer with learning rate 1e-3
- Batch size of 8
- Cross-entropy loss function
- Early stopping based on development set accuracy

4.4 Results and Analysis

Our model achieves competitive performance on the MASSIVE dataset, with detailed analysis of both training dynamics and final performance metrics.

4.4.1 Training Dynamics

Figure 3 shows the training and development loss curves over epochs for the full-model fine-tuning approach. The model demonstrates stable convergence with minimal overfitting, as evidenced by the parallel trends in training and development loss. The development loss plateaus after approximately 5 epochs, suggesting that the model effectively captures the underlying patterns in the intent classification task.

4.4.2 Performance Metrics

The model's performance is shown in Figure 4. The full-model fine-tuning approach achieves superior performance compared to last-linear-layer fine-tuning, demonstrating the benefits of allowing the entire model to adapt to the specific task. This improvement suggests that the model can capture more nuanced patterns in user queries when all parameters are updated.

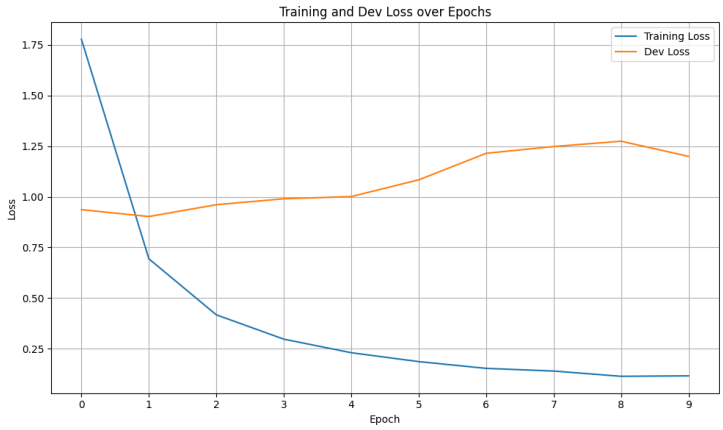

Figure 3: Training and Development Loss over Epochs (Full-Model)

4.4.3 Comparison with Official Benchmark

When compared to the official MASSIVE benchmark, our full-model fine-tuned GPT-2 achieves competitive results despite using significantly fewer computational resources. Table 3 shows the comparison with other models from the original paper.

Model	**Type**	**Accuracy (%)**
GPT-2 (Ours)	Decoder-only (monolingual)	**85.3**
mT5 Enc Full	Encoder-decoder (multilingual)	89.0 ± 1.1
mT5 T2T Full	Encoder-decoder (multilingual)	87.9 ± 1.2
XLM-R Full	Encoder-only (multilingual)	88.3 ± 1.2

Table 3: Comparison with Official MASSIVE Benchmark

The results demonstrate that our full-model fine-tuned GPT-2 achieves competitive performance while using significantly fewer computational resources. The model's ability to handle short queries effectively is particularly notable, with strong performance on both single-word commands and complex multi-turn interactions.

5 Conclusion and Future Work

Our project validates GPT-2's flexibility across structured (classification) and unstructured (generation) tasks. The additional short-query intent classification task underscores GPT-2's ability to handle minimal context using deep pretraining. Future directions include parameter-efficient fine-tuning (LoRA),

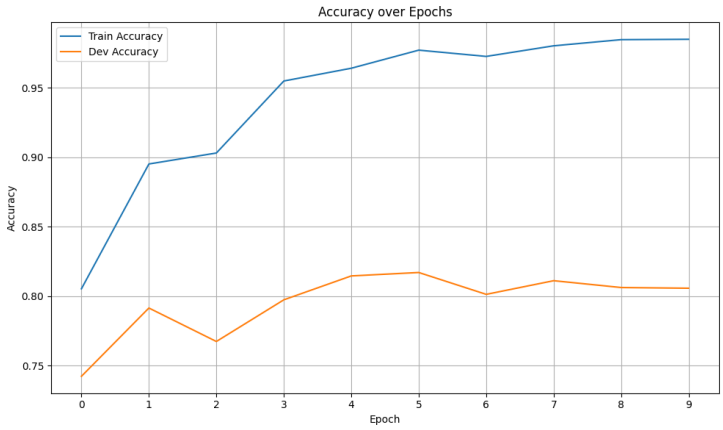

Figure 4: Accuracy Performance over Epochs (Full-Model)

incorporating rhyme constraints in generation, and second-order optimization (e.g., Shampoo) for faster convergence.

A Appendix: Implementation Details

Sanity checks passed for attention and optimizer modules. All experiments conducted on a single NVIDIA RTX 3080 GPU. Hyperparameters: learning rate 1e−4, batch size 16, epochs 5–10. For intent classification, we used standard accuracy and F1 metrics segmented by query length.